Stalin: The Game

Robin Langer

1 September 2026

1 The Unix philosophy, pipes, and CLI tools

In case the reader is a little rusty, we begin with a brief review of the Unix this note leans on: what a process is, how one reaches the world outside itself, how two of them are joined together, and why a command-line tool has the shape it has. None of this is specific to the game `stalin`, but all of it is used by the game `stalin` — so the review is the first half of the explanation of our design choices.

Each part below introduces one idea and, where it helps, one small example that illustrates only that idea. Nothing is used before it has been introduced.

A process

A *process* is one running program, together with everything the kernel keeps on its behalf: its memory, its position in the code, its identity — which user it runs as — and a number. A program is a file sitting on a disk, and does nothing; a process is that file in motion. Run the same program twice and you have two processes which share nothing and cannot see each other.

The number is the process's *pid*. It is how anything outside the process refers to it, and there is no other handle: to stop a program you send a signal to its pid.

User space and kernel space

A running machine is divided in two. Your code executes in *user space*, where it may compute freely and touch nothing: it cannot address a disk, a network card, or another process's memory, because the processor is in a mode that forbids it. The *kernel* executes in kernel space, where it may touch everything. The division is enforced by the hardware, not by convention.

There is exactly one interface between the two: the *system call*. It suspends your process, switches the processor into kernel mode, runs kernel code on your process's behalf, and returns. Every effect a program has on the world outside its own memory is a system call. There is no other mechanism.

`read` and `write`

That interface could have been wide. It is in fact very narrow. The two calls that matter most are these:

```
read (n, buffer, count)   -- give me up to count bytes from the thing numbered n
write(n, buffer, count)   -- take these count bytes and put them in the thing
    numbered n
```

Each takes a number identifying something already open, a place to put bytes or take them from, and how many. Between them they are very nearly the whole interface to the outside world. What that number is, and where it comes from, is the subject of two subsections below.

Everything is a file

Why should two calls be enough? Because of a design decision usually stated as a slogan: **everything is a file**. A file on a disk, a terminal, a network connection, a source of random bytes — all of them are presented to user space through the same interface, so the same two calls reach all of them.

The slogan overstates itself in one respect, and it is worth being precise about which. Things do differ in how they are *made*: a file is opened by name, a network connection has to be built by a short sequence of calls of its own. What they do not differ in is how they are *used*. Once the thing exists and you are holding its number, `read` and `write` are the whole vocabulary. The uniformity is at the point of use — which is the point that matters, because using is where one program meets another.

File descriptors

That number is a *file descriptor*. It is a small non-negative integer, and it is an index into a table the kernel keeps for each process, whose entries point at the things that process has open. Nothing in the number says what kind of thing it is. The descriptor is an opaque handle, and that is exactly why one program can be pointed at a file one day and at a network connection the next without being rewritten.

Three entries are filled in before a process begins, and it did not fill them in itself — whoever started it did.

```
0   standard input    the stream the program reads
1   standard output   where its results go
2   standard error    where its complaints go
```

Start a program from a terminal and all three point at that terminal, which is why results and complaints appear jumbled together in one scrolling window. That is a property of the terminal, not of the program.

`cat`

Everything so far is enough to write a real Unix program. `cat` — named for *concatenate* — copies its input to its output, unchanged, and does nothing else. That is its entire specification, and it is almost exactly `read` and `write` in a loop:

```
char buf[4096];
for (;;) {
    n = read(0, buf, 4096);   -- take up to 4096 bytes from standard input
    if (n == 0) break;        -- 0 bytes means end of input; stop
    if (n <  0) exit(1);      -- negative means the read itself failed
    write(1, buf, n);         -- put exactly those n bytes on standard output
}
```

The real `cat` is longer, but the extra length is all flags and error messages; that loop is the program. Look at what it does *not* contain. It never asks what `0` is attached to, or what `1` is attached to, because `read` and `write` provide no way to ask. A disk file, a keyboard, another program's output: the loop is identical and the program cannot tell the difference.

`fork` and `exec`

Two more system calls, and they are the two that make new processes.

`fork` *duplicates* the calling process. Where there was one process there are now two, identical in their memory, their open descriptors and their position in the code, differing only in the value `fork` handed back — which is how each tells whether it is the original or the copy.

`exec` *replaces* the program running inside a process. Same process, same pid, same descriptor table, but the old code and memory are discarded and a new program is loaded in their place.

Neither is a "run this program" call, and that is the point: to start a program, a process `fork`s, and then the copy `exec`s. The descriptor table survives both, so the copy inherits the original's descriptors — and in the gap between the two calls the copy is still running the original's code, so it can rearrange its own table before replacing itself. Three subsections below turn on that gap.

The shell

The shell is not part of the kernel and has no privilege you do not have. It is an ordinary program running in user space with your identity, and its purpose is to start other programs. That is all it is: a program whose job is running programs. Two things it relies on come first, and then what it does with a line you have typed.

The environment

One more thing a process carries: its *environment*, a set of named strings held alongside its memory and its descriptor table. They are called variables, but they are only ever text. Like the descriptor table, the environment survives `fork` and `exec`, so a new process starts with a copy of the one that started it:

```
$ export GREETING=hello     -- add it to this shell environment
$ sh -c 'echo $GREETING'    -- a different process, started by this one
hello
```

That is the whole of it: named text, inherited downwards. The most used of them is `PATH`, which holds a list of directories; the shell searches them whenever it is looking for a program to run.

Globbing

A word containing `*`, `?` or `[…]` is a *glob*: a pattern to be matched against the names of files that exist. `*` stands for any run of characters, `?` for a single one, `[abc]` for one character out of a set.

The matching is done by the shell, before any program is started:

```
$ echo *.txt
log.txt notes.txt other.txt
```

`echo` prints its arguments and nothing else, so what it printed is exactly what it was given: three filenames. It never saw an asterisk. That is why every program accepts patterns, and why no program contains any code for matching them.

What the shell does with a line

Given a line, the shell does five things, and every one of them has already been described.

1. **Split the line into words.** The first word names what to run; the rest become the argument vector that program will receive.
2. **Find the program.** The first word is looked for in each directory named by `PATH`, in order, and the first match wins.
3. **`fork`.** The shell duplicates itself.
4. **`exec`.** The copy replaces itself with the program. The shell that read the line is still there, waiting for it to finish.
5. **Keep the exit status.** When the program finishes the kernel gives the shell a small number it left behind, by convention zero for success.

One command cannot work that way, and the reason is worth seeing. `cd` has to be built into the shell rather than being a program, because a program runs in the copy, and the copy changing its own working directory would leave the shell exactly where it was. Anything that must alter the shell's own state has to be part of the shell.

A remark, because this is the mechanism behind something that catches people out with coding agents. Claude Code runs each shell command exactly as the shell runs a program: it forks, and the copy execs. A `cd` in that command changes the copy's working directory, and the copy then exits. So no command the agent runs can change the agent's. The directory it was launched in is therefore the directory it has for the whole session, which is why you must start it in the project you actually mean to work on.

Redirection

Now the gap between `fork` and `exec`. Writing `> log` after a command tells the shell: in the copy, and before exec'ing, open the file `log` and put it in slot 1.

```
$ echo hello > log      -- slot 1 is now the file, not the terminal
$ cat log
hello
```

`2> errs` redirects descriptor 2 in the same way. Nothing was added to `echo` to make this work, and `echo` cannot discover that it happened: it wrote to descriptor 1 exactly as it always does. Redirection is the shell's syntax and the shell's doing, from beginning to end.

Pipes

A *pipe* is a buffer the kernel will hand out on request, with two descriptors: one to write into, one to read out of. `|` is the same redirection as `>`, with a pipe where the file was. The shell asks for a pipe, forks twice, puts the writing end in the left program's slot 1 and the reading end in the right program's slot 0, and execs both.

```
$ cat notes.txt | wc -l
       3
```

`wc -l` counts the lines on its standard input. It was given no filename and does not know one exists.

Both programs run at the same time; `wc` does not wait for `cat` to finish. The buffer is finite — 64×1024 bytes on Linux — and the kernel makes each side wait for the other rather than growing it. When it is full, `cat`'s next `write` does not return until `wc` has consumed something; when it is empty, `wc`'s next `read` does not return until `cat` has produced something. Neither program contains a line of code about this.

Two things follow, and both are worth having in mind. A pipeline over a very large file needs no more memory than a pipeline over a small one, because at no moment does either process hold more than the buffer. And the second program starts answering immediately rather than at the end: in `cat huge.log | grep x`, the command `grep` is handed the first 64×1024 bytes as soon as `cat` has written them and reports its first match from those, while `cat` is still reading. It does not need to wait for `cat` to finish, and nothing in either program arranges for it not to. That is simply what two processes and one finite buffer do.

Exit status, and the shell's control flow

The small number a program leaves behind is what the shell builds its control flow on.

- `;` — run the next command regardless of the status.
- `&&` — run the next command only if the previous one exited zero.
- `||` — run the next command only if the previous one exited non-zero.
- `&` — start this command and do not wait for it, so several can run at once.

We could say a good deal more about all this, but it is not really relevant to the game `stalin`.

Shell scripts

A file of such lines is a shell script, and running it is the same as having typed them. So a pipeline that worked once can be kept and named. And because a script is started like any other program — found on `PATH`, `fork`ed, `exec`ed, its status collected — a script can appear in a pipeline itself, on either side, and nothing that uses it needs to know it is a script.

Three channels, not one

Gather up what a program actually offers the world outside it. There are three separate channels, and they carry three different kinds of thing.

Channel	Carries	Read by
descriptor 1 (stdout)	the results	the next program in the pipeline
descriptor 2 (stderr)	the narration	a person
the exit status	the verdict	the shell

Keeping them apart is what makes a program usable by another program: a pipe takes descriptor 1 and nothing else, so whatever reads from a program receives its results and never has to sort them out from error text. Putting results and narration both on descriptor 1 is the commonest way to make a program unusable in a pipeline.

Manual pages, and what is not checked

None of the above is checked by anything. The interface between two programs is bytes: any two can be joined, which is the strength, and nothing verifies that the composition means anything, which is the cost. What a program promises is written in its manual page and nowhere else, so the only thing enforcing it is that you read it.

Here is what that costs. You want to build only if the log has no errors in it, so you write this:

```
$ grep -c ERROR log && ./build
```

`&&` runs the next command only if the previous one exited zero, and `grep` exits zero when it *finds* something. So the line does the exact opposite of what you wanted.

```
$ grep -c ERROR dirty.log && ./build
1
  building                  -- errors present, grep exits 0, so it builds
$ grep -c ERROR clean.log && ./build
0                           -- no errors, grep exits 1, so it does not
```

It builds exactly when the log is dirty and refuses exactly when the log is clean. Both halves of `grep`'s behaviour are documented and neither is enforced, the shell did precisely what it was told, and nothing anywhere reports a problem.

What you wanted was `grep` answering *did you find anything* rather than *how many*. That is what `-q` is for: it prints nothing at all and reports only in its exit status — zero if it found at least one match, one if it found none. Put the test where the shell can actually see it, and use `||` so the build runs when nothing was found:

```
$ grep -q ERROR log || ./build
```

Notice that `-c` was carrying that same exit status all along. The count it printed was never the thing `&&` was reading.

A note before leaving this. Claude is an absolute master at writing shell scripts, and this is genuinely a place where you do not need the details. Ask Claude for a script that builds only when the log is clean and you will get a correct one. What you do need is the concept.

None of this is enforced, and this is a deliberate design decision. The only thing holding any of it together is "developer discipline" — somebody read the manual page — and at this level of the stack that is the whole of the enforcement there is.

The dictum

Let's summarise what we have: one uniform interface reached by `read` and `write`, three separate channels, and a pipe that joins one program to another. With that on the table, the philosophy reads as a consequence of the machinery rather than as an aspiration. Doug McIlroy, who wrote the pipe into Unix v3 in 1973, put it in three sentences:

> Write programs that do one thing and do it well. Write programs to work together. Write programs to handle text streams, because that is a universal interface.

Mike Gancarz later expanded this into nine tenets — small is beautiful; each program does one thing well; build a prototype as soon as you can; choose portability over efficiency; store data in flat text; use software leverage; use scripts to increase leverage and portability; avoid captive user interfaces; make every program a filter.

The last is the one that matters here, and it is the least obeyed. A *filter* is a program that reads a stream and writes a stream: its input is not a dialogue and its output is not a report to a human. `cat`, `wc` and `grep` are filters, which is why the examples above could be assembled without any of them having been designed for each other.

A CLI tool is already a dependent function

One last observation, and it is the one this note turns on.

A command-line tool receives one prompt — its argument vector — and produces a response whose *type depends on which prompt it was*. `git log` answers with a list of commits; `git status` answers with a working tree. Those are not the same kind of thing, and which kind you get is fixed by the word after `git`. That is $(p : P) \to Rp$, with P the subcommands and R the result type of each. The interface (P, R) is what `--help` prints.

Notice what is *absent*. No event loop, no callbacks, no waiting, no concurrency. A CLI starts, is prompted once, answers, and exits. It is nonetheless event-driven in the sense that matters, because what makes a program event-driven is the prompt-indexed response type and not the waiting. Strip the loop away and this is what remains.

2 The Claude Code harness: kernel, shell, and permissions

What the harness is

Claude Code is a CLI process. When it runs a command it does what a shell does: it forks, it execs, and the child inherits the parent's identity. There is no sandbox in that sentence and no daemon and no privileged helper. **Claude Code can execute any command-line tool with the permissions of the user who launched it** — on this machine, `robin`. Every command in this note ran as `robin`, the servers included.

That is worth stating baldly because it is the security model, and it is a model people routinely mis-describe. An agent deleting your home directory is not privilege escalation. It is your own privilege, used badly, by a process you started. Unix gives every process *ambient authority*: authority attaches to the identity of the running user, not to the particular task, so a process may do anything its user may do merely by virtue of existing. There is no way, at the kernel level, to say "this program may read *these* files" — only "this user may read these files".

3 HTTP and JSON

Again, in case the reader is a little rusty, we review some concepts from web app development: what HTTP is and why so little of it turned out to be the right amount, what a status code is for, which part of a request carries what, and what JSON does and does not preserve. This time the review and the explanation are interleaved, because from here on the concepts are put to work on the code as they arrive.

A design that looked far too weak

HTTP is a request/response protocol: a client sends a request, a server sends a response, and nothing else happens. A request is a method, a path, some headers and optionally a body; a response is a three-digit status, some headers and a body. It is worth a paragraph on where so little came from, because HTTP made the same kind of bet Unix made, and it looked just as inadequate at the time.

Tim Berners-Lee's first version, at CERN in 1991, had one method. You opened a connection, sent a single line — `GET /thepage` — and read the document that came back. There were no headers, no status codes, and no content types. The end of the response was signalled by the connection closing. That is the whole protocol. Methods, statuses and headers arrived in 1996, and persistent connections in 1997, but the shape did not change: one request, one response, and a server that remembers nothing about you afterwards.

Statelessness in particular looked like a straightforward defect. A protocol that cannot remember the previous request cannot have sessions, cannot have transactions, cannot notify you when something happens, and cannot check that what you sent was even the right kind of thing. The competition was much better appointed. Gopher, of the same year, served structured menus rather than untyped documents. The distributed-object systems of the decade — CORBA, then DCOM, later SOAP — offered typed interfaces, described in a formal language, with generated client stubs and objects on the server that stayed alive between calls. On paper they were strictly more capable, and almost all of it is gone.

What the weakness bought is exactly what it looks like it costs. Because a request carries everything needed to answer it, *any* server can answer *any* request: put a hundred machines behind one name and requests may be sprayed across them in any order. Because a response depends only on its request, anything in between may keep a copy and answer the next one itself. And because the server holds nothing on your behalf, a server that dies loses one request rather than your session. Load balancing and caching were not added to HTTP later — they are consequences of the thing that looked like the flaw.

The best evidence is what has happened since. The encoding on the wire has been replaced twice — a binary framing in 2015, and a different transport underneath it in 2022 — and both times the semantics were left exactly as they were: a method, a path, some headers, a body, and a three-digit status. Two complete rewrites of how the bytes travel, and nothing above had to be rethought. That is what a correct interface looks like in retrospect.

The parallel with the previous section is close enough to be worth naming. A pipe carries bytes and does not know what they mean; an HTTP response carries bytes and a string saying what they are. In both cases the interface is universal precisely because it is weak, in both cases nothing verifies that an exchange means anything, and in both cases what was rebuilt on top of the weakness — sessions, from cookies onwards — had to be rebuilt without any help from the protocol at all.

Why HTTP at all

The five commissariats could have been five modules and every hop a function call. They are not, and the reason is in the comment at the top of `wire.ts`:

> A hop costs a process, which is the whole point: control genuinely flows down, and the boundary is real rather than a function call wearing a costume.

A container morphism has a delegate leg $u : P \to Q$ that maps *shapes*, and shapes are exactly what survives a wire. If the hop were a function call, nothing would stop a handler reaching into the caller's state, and the claim that the levels are separate would be untestable. Making it a socket makes the claim falsifiable: anything that crosses must be expressible in JSON, and the compiler on the far side has never heard of your types.

The cost is honest and is written down — about a fifth of a second per hop, which a hundred-game parameter search pays forty times per game, hence the `STALIN_INPROC` escape hatch that keeps the HTTP hop and skips only the spawn. No game a player sees uses it.

Status codes carry the kind of failure

This is the part worth copying into other projects. The three outcomes are genuinely different kinds of thing, and the status vocabulary already distinguishes them. From `serveContainer`:

```
try { body = await req.json(); }
catch { return json({ error: "not JSON" }, 400); }

const prompt = opts.parse(body);
if (prompt === null) {
  // The validator refused.  No cast was made, so nothing ill-typed is
  // downstream of this line -- that is the whole point of validating.
  return json({ error: "not a valid prompt for this container", got: body }, 400);
}
try   { return json(await opts.answer(prompt, depth + 1), 200); }
catch (e) { return json({ error: msg }, 500); }
```

Against the live transport commissariat on port 8804:

```
$ curl -s -i -X POST localhost:8804 -H 'content-type: application/json' \
    -d '{"tag":"Haul","railCapacity":40,"available":100,
         "needIndustry":30,"offerPort":50,"year":1929}'
HTTP/1.1 200 OK
{"toIndustry":30,"toPort":10,"stranded":40,"short":0}

$ curl -s -X POST localhost:8804 -d '{"tag":"Haul","railCapacity":"lots"}'
HTTP/1.1 400 Bad Request
{"error":"not a valid prompt for this container","got":{"tag":"Haul","railCapacity
    ":"lots"}}

$ curl -s -X POST localhost:8804 -d 'not json at all'
HTTP/1.1 400 Bad Request
{"error":"not JSON"}

$ curl -s -o /dev/null -w '%{http_code}\n' -X POST localhost:8804 -d '{"tag":"Purge
    "}'
400
```

Read the 200 for a moment, because it is the game in four numbers. Forty units of rail capacity, a hundred units of grain in hand, thirty needed at the mill and fifty offered to the port: the mill is fed first, which leaves ten units of capacity, so ten reach the port and **forty tons are stranded** — grain the state owns, has procured, and cannot move. That is why a railway is throughput and not capital: build too little of it and the grain rots where it stands, whatever the harvest was.

The three 400s are all *"that was not a prompt"*, at three depths: not JSON at all; JSON but not this container's shape; a tag this container does not have. Only the 500 means "that was a prompt and answering it went wrong". Getting this split right is what lets a caller distinguish a bug in itself from a bug in its callee, which is the only reason status codes exist.

Where the depth rides, and why

`wire.ts` opens by saying two things cross between levels: a JSON document and a depth counter. The depth is used only for indenting the trace. It travels in a header:

```
const DEPTH_HEADER = "x-tower-depth";

export const depthHeaders = (depth: number) => ({
  "content-type": "application/json",
  [DEPTH_HEADER]: String(depth),
});
```

The reason is a one-liner in the source and it is a genuine design principle:

> The depth rides in an HTTP header rather than in the prompt, because the prompt is the container's shape and nothing that isn't a prompt belongs in it.

HTTP offers three places to put data — query string, headers, body — and they are not interchangeable. The body is the container. The header is everything true about the *message* rather than about the prompt. Had the depth gone in the body, every shape type in the repository would have acquired a field that means nothing to the game, and the shape types would have stopped being the containers' shapes, which is the one correspondence the whole exercise rests on.

What a hop is, on the wire

`lib/delegate.ts` is forty lines and does one thing: it carries a JSON prompt to a port and prints the reply. It knows nothing about the game, nothing about containers, and nothing about what any of the fields mean.

```
$ deno run --quiet --allow-net --allow-env \
    lib/delegate.ts 8804 '{"tag":"Census"}' 0
{"workforce":{"fields":0,"drivers":0,"mill":0,"rail":0,"dead":0},
 "stocks":{"railCapacity":0},"trueOutput":0}
```

The three channels are kept strictly apart, and you can check it:

```
$ deno run ... lib/delegate.ts 8804 'garbage'          0 ; echo $?    # 2
$ deno run ... lib/delegate.ts 9999 '{"tag":"Census"}' 0 ; echo $?    # 1
$ deno run ... lib/delegate.ts                             ; echo $?    # 2
```

The first is a prompt that is not JSON and the third is no arguments at all — both `2`. The second is a well-formed prompt fired at a port with nothing behind it — `1`. So there are two exits from failure and they mean *different* things: `2` means *you* were wrong, `1` means the world was.

Every diagnostic goes to descriptor 2, and descriptor 1 carries the reply and nothing else. That is what lets the caller parse the whole of stdout as one JSON document without stripping anything off it first. The comment in the source says it in a line: *stdout is the reply and nothing else, so the parent can JSON.parse it whole.*

The same discipline is what makes the tracing work. `lib/wire.ts` writes its ↓/↑ trace to stderr:

```
export function trace(depth: number, dir: "down" | "up", who: string, what: string)
    {
  const line = `${"  ".repeat(depth)}${dir === "down" ? "↓" : "↑"} ...`;
  Deno.stderr.writeSync(enc.encode(line));   // stdout stays pure JSON
}
```

So a running game narrates itself on descriptor 2 while descriptor 1 stays machine-readable. The block below is assembled from the five servers' separate logs; with their stderr inherited instead, the children's trace lines join the parent's directly.

```
↓ gosplan   Sow order={"procurement":"firm","labour…":{"
  ↓ gosplan   ((agri (x) smelter) (x) transport)
    ↓ agri      Harvest workforce={"fields":100,...} tractors=0 year=1928
    ↑ agri      {grain, grade, perWorker, withoutTractors}
    ↓ industry  Smelt workers=12 millCapacity=8 year=1928
    ↑ industry  {steel, idleCapacity}
↑ gosplan   {kind, year, harvest, steel, …}
```

Indentation is the depth counter; ↓ is control flowing down and ↑ is data flowing back up. You are reading the derivation tree of a container morphism, printed by a program that had no idea it was doing that.

JSON, and what it throws away

JSON has six types: objects, arrays, strings, numbers, booleans, null. That is the whole grammar, and the omissions are what matter at a typed boundary.

- **No sum types.** A constructor becomes a string field by convention. The `"tag"` in every prompt is that convention, and it is *a claim, not a fact*: nothing in the wire format prevents `{"tag":"Haul"}` with no other field, which is exactly the 400 above.
- **No functions.** This one bites structurally. The shape of $\triangleleft$ is a dependent pair whose second component is a function `(r: PA) => SB` — and a function cannot be serialised. So sequential composition is necessarily resolved *above* the wire, in the runner, which prompts the first container and applies `next` to the reply to obtain the second prompt. Only the two ground prompts ever cross. The wire format is not neutral about which combinators can be distributed.
- **One number type,** IEEE 754 double. Grain and steel are tonnages, so this is fine here, and it would not be fine for money.
- **No distinction between absent and null,** and no schema of any kind.

Hence the shape of every crossing, in both directions: `unknown` in, and a witness required before anything else may happen. Which is §4.

Statelessness, and where this repository is not stateless

`stalin` does not build the sessions-on-top-of-cookies machinery described above; it keeps the state in the process, at module scope:

```
let seed  = Number(Deno.env.get("STALIN_SEED") ?? "1928");
let books = new Books("Narkomput", seed + 4, 0.4, 0.92);
let laid  = 0;
let lastQuota = 0;
```

This is a deliberate and defensible choice — a commissariat *is* a stateful thing, and the point of the exercise is the container algebra, not session management — but be clear about the consequence, because it is a real one. The protocol cannot clear that state, so clearing it has to become a move in the game:

```
Reset: (p) => { seed = p.seed; books = new Books(...); laid = 0; lastQuota = 0;
                return { ok: true }; },
```

`Reset` is a prompt with no in-fiction meaning; no plan resets a railway. It exists because `stalin new` must reach the commissariats, or the weather stream and the books carry over into the next game — a bug the comment records as having actually happened. Determinism on the seed is what makes a playthrough a regression test, so this is load-bearing. The general lesson: when process state substitutes for protocol state, the lifecycle leaks into the interface.

4 TypeScript, event handlers, and servers

What TypeScript is, exactly

TypeScript is a structural type system over JavaScript that is *erased*. Types are checked at compile time and then deleted; the emitted program has no representation of them and no run-time check derived from them. There is no reflection to fall back on. A cast is therefore not a coercion — it is an unchecked assertion, a promise to the compiler that the compiler does not verify and the runtime cannot.

That single fact determines the shape of every boundary in this codebase. On the inside, the checker is strong enough to do real work. A conditional type over a finite union *computes* which decisions exist in each case — here, which export decisions the harvest permits:

```
type ExportChoice<G extends HarvestGrade> =
  G extends "failure"  ? "none"
: G extends "poor"     ? "none" | "surplus"
: G extends "adequate" ? "none" | "surplus" | "full"
:                        "none" | "surplus" | "full" | "maximum";

exportPlan("bumper",  "maximum")   // fine
exportPlan("failure", "maximum")   // does not compile
```

Exporting grain during a failed harvest is not a move the engine rejects. It is a move that cannot be written down. On the outside, at the wire, the checker contributes exactly nothing, because everything arriving is `unknown` and the checker went home before the process started.

Servers: the runtime owns the loop

`Deno.serve` takes a handler and supplies everything else — the socket, the accept loop, the parsing, the concurrency:

```
Deno.serve({ port: opts.port }, async (req) => { ... return response; });
```

You write (Request) → Promise⟨Response⟩ and the runtime calls it once per request. This is what "event handler" means operationally: inversion of control, with the loop on the far side of the library boundary.

But — and this is the §1 point returning — the loop is not what makes it event-driven in the sense that matters. The CLI in `stalin.ts` has no loop at all and is event-driven in the same sense. What both have is a response type indexed by the prompt. The loop is scheduling; the indexing is the type.

The handler table is the dependent function

Here is the pattern used in all five commissariats, from `transport.server.ts`. The prompt type is a tagged union — a finite shape type:

```
export type TrPrompt =
  | { tag: "Lay";    workers: People; steel: Steel; year: number }
  | { tag: "Haul";   railCapacity: number; available: Grain;
                     needIndustry: Grain; offerPort: Grain; year: number }
  | { tag: "Report"; year: number }
  | { tag: "Census" }
  | { tag: "Reset";  seed: number };

export interface TrResponses {
  Lay: RailReport;  Haul: HaulReport;  Report: Dispatch;
  Census: CensusReturn;  Reset: ResetAck;
}
type Reply<K extends TrPrompt["tag"]> = TrResponses[K];
```

`TrResponses` *is* $R : P \to \mathrm{Type}$, written as a record because the index is finite. And then the program $(p : P) \to R\,p$ is a record too:

```
type Handlers = { [K in TrPrompt["tag"]]: (p: Extract<TrPrompt, { tag: K }>) =>
    Reply<K> };

const handlers: Handlers = {
  Lay:  (p) => { ... return { added, steelUsed }; },
  Haul: (p) => { ... return { toIndustry, toPort, stranded, short }; },
  ...
};
```

Three properties are worth pausing on, because this is the idiom to internalise:

1. **Each branch is checked against its own fibre.** Inside `Haul`, `p` is the `Haul` member and the return type is `HaulReport`. Not a union, not a widened supertype — the one type indexed by that tag.
2. **Totality is structural.** Add a member to `TrPrompt` and the object literal stops satisfying `Handlers`. There is no default case to fall through and no runtime dispatch to forget. Exhaustiveness is a property of the mapped type, not of anyone's diligence.
3. **Ordinary code, dependent shape.** No dependent types were harmed. This works because the index is a finite union of string literals, over which conditional and mapped types distribute — the same observation that makes `lib/algebra.ts` possible, where a container is presented as its graph, $\Sigma(s{:}S).\,B\,s$ as the union of its fibres. What it costs is a `next` that branches between *different* second shapes: such a function has a union return type, lies in no single fibre, and cannot be written in this encoding at all.

The one cast, and why it is there

Applying the table needs a cast, and the codebase confines it to three lines:

```
function answer<P extends TrPrompt>(p: P): Reply<P["tag"]> {
  const h = handlers[p.tag] as (q: TrPrompt) => Reply<P["tag"]>;
  return h(p);
}
```

This is the well-known correlated-union limitation: TypeScript will not see that `handlers[p.tag]` and `p` are correlated through the same tag, so it types the lookup as a union of functions and the argument as a union of parameters, and refuses the application. The cast asserts the correlation that the mapped type already established one declaration earlier. It is unchecked, and it is safe for a reason you can state: every entry of `handlers` was individually checked at its own fibre when the literal was written. Note the discipline — one cast, at a place where the invariant is provable, not scattered through the handlers.

The seam: validate, do not cast

Now the boundary, which is the most important twenty lines in the repository. Types die at the wire. Something must witness them back into existence, and the only honest witness is code that looks at the fields.

```
export function parseTr(u: unknown): TrPrompt | null {
  if (!isRecord(u) || !isStr(u.tag)) return null;
  switch (u.tag) {
    case "Haul":
      return isNum(u.railCapacity) && isNum(u.available) && isNum(u.needIndustry)
    &&
             isNum(u.offerPort) && isNum(u.year)
        ? { tag: "Haul", railCapacity: u.railCapacity, available: u.available,
            needIndustry: u.needIndustry, offerPort: u.offerPort, year: u.year }
        : null;
    ...
    default: return null;
  }
}
```

Four things are being done deliberately here.

The signature is the whole design. `unknown → TrPrompt | null`. Not `any`, which would let the ill-typed value spread silently; `unknown`, which the compiler will not let you touch until you have narrowed it.

Refusal is a value, not an exception. `null` means "this was not a prompt". The caller in `serveContainer` turns it into the 400 of §3. An exception would have conflated it with the 500, which is the distinction the status codes exist to draw.

The tag is checked, not trusted. `u.tag === "Haul"` says the sender claims this is a `Haul`. The five `isNum` calls are what make it one. Everything arriving from outside is a claim rather than a fact, and that holds for your own five servers on localhost too, because the property you want is not "my callers are honest" but "an ill-typed value cannot get past this line".

The object is rebuilt, not passed through. The returned literal names every field explicitly. Nothing extra rides along, and the returned value is a `TrPrompt` because it was *constructed* as one, not because anyone asserted it.

Replies coming back are validated the same way, by per-tag decoders in `lib/algebra.ts`:

```
export type Decoders<C extends Cont> = {
  [K in TagOf<Shape<C>>]: (u: unknown) => PosAt<C, Extract<Shape<C>, { tag: K }>> |
     null;
};
```

Same mapped-type idiom, pointed the other way: one decoder per shape, each checked against that shape's own position type. A leaf runs by delegating and then decoding, and throws if the reply does not validate. So both legs of every hop have a witness, and there is no direction in which an unchecked value enters the typed world.

Three roles in one process

Finally, notice what a commissariat is. The same process:

1. **listens** — `serveContainer` on its port;
2. **calls** — `fetch`, when its handler needs a lower level;
3. **spawns** — `new Deno.Command("deno", ...)`, because a hop is a process.

That third role is also what makes the tower a tower rather than a star: gosplan is a client of agriculture, which may in turn be a client of something else, and the depth header counts the storeys. `await` is what joins the two directions — control flowing down the stack of processes, data flowing back up — and the indentation in the trace shown earlier is that stack, printed.

Summary

One shape recurs at every level of this stack: **a prompt determines the type of its response**. What changes from level to level is only who enforces it.

Level	Prompt	Response	Enforced by
a pipe	the bytes upstream	the bytes downstream	nobody
a CLI tool	the argument vector	stdout + exit status	a man page
a syscall	`read`/`write` on an `fd`	bytes, or an error	the kernel
an HTTP request	method, headers, body	a status and a body	the status code
a handler	a tagged prompt	the reply at that tag	the compiler

The enforcement gets stronger as you read down that table, and it also gets narrower: the kernel checks everything and understands nothing, the compiler understands everything and checks only what never left the process. Neither end can be dispensed with, so the engineering is all in the handover, and there are two places in this repository where it happens.

- `lib/delegate.ts` is the handover from §1 to §3. A pipeline's untyped byte stream becomes an HTTP request, and the discipline that makes it work is entirely Unix's: one job, stdout is the reply and nothing else, stderr for narration, two distinct exit codes for two distinct kinds of failure.
- `parse` is the handover from §3 to §4, and it is the important one. `unknown → TrPrompt | null`: refusal is a value, the tag is checked rather than trusted, and the prompt is rebuilt field by field rather than asserted. Nothing ill-typed exists downstream of that line, and it is the only reason the mapped-type machinery above it means anything at run time.

The type system's contribution is real but strictly local. It computes which moves exist in which case, and it makes a handler table total by construction, so a whole class of mistake becomes unwritable rather than merely untested. It contributes nothing at the wire, where everything arriving is `unknown` and the checker has already gone home. A single unvalidated cast at that boundary would forfeit every guarantee above it — which is why there is exactly one cast in this codebase, three lines long, in a place where the invariant it asserts was established one declaration earlier.